Operatori e stampa a video

Lezione 4 — 1IOT, SST-ICT-01 — Operatori aritmetici, modulo, assegnazione, printf

21/09/2026

1 Ripasso alla lavagna: i tre tipi di dato

Prima di introdurre gli operatori, un breve richiamo grafico di quanto visto la lezione scorsa, così come è stato scritto alla **lavagna**:

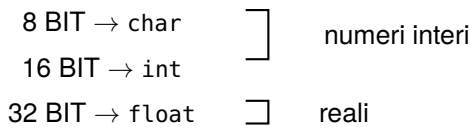


Figura 1: Come scritto alla lavagna: char e int raggruppati come "numeri interi", float isolato come tipo per i "reali" (i numeri con la virgola).

2 Cosa sono gli operatori

Definizione

Un **operatore** è il **simbolo** che si mette tra due o più **operandi** (elementi) per **combinarli** e ottenere un **risultato**. In base al simbolo usato, si esegue un'**operazione diversa**.

Esempio

In matematica, $5+2=7$: prendo gli elementi 5 e 2 e li combino con l'**operatore** più, ottenendo 7. Se li combino con la **moltiplicazione** ottengo invece 10: stesso operandi, operatore diverso, **risultato diverso**.

3 Gli operatori aritmetici

Definizione

Gli **operatori aritmetici** (o matematici) sono quelli imparati alle elementari: `+` (**somma**), `-` (**sottrazione**), $*$ (**moltiplicazione**, asterisco), `/` (**divisione**, chiamato *slash*).

Nota

In informatica la **divisione** si scrive con lo **slash** (`/`), non con i **due punti** o il simbolo della tabellina usato in matematica.

3.1 L'operatore modulo

Definizione

Il simbolo `%` (che sembra una **percentuale**, ma **non lo è**) si chiama **operatore modulo**: restituisce il **resto** di una **divisione tra numeri interi**.

Esempio

Quanto fa $5 \div 2$ tra interi? Il risultato della divisione è 2, con **resto 1**. Quindi $5 \bmod 2 = 1$.
Quanto fa $11 \bmod 3$? $11 \div 3 = 3$ (perché $3 \times 3 = 9$), e $11 - 9 = 2$: quindi $11 \bmod 3 = 2$.
Quanto fa $100 \bmod 30$? $100 \div 30 = 3$ (perché $3 \times 30 = 90$), e $100 - 90 = 10$: quindi $100 \bmod 30 = 10$.
Altri esempi fatti alla lavagna: $16 \bmod 5 = 1$, $40 \bmod 9 = 4$.

Attenzione

Il modulo dà **solo il resto**, non il **risultato della divisione**. Per il valore della divisione serve lo **slash** (`/`). Comprendere bene questo operatore è **fondamentale**: verrà usato moltissimo più avanti nella programmazione.

4 L'operatore di assegnazione

Definizione

Il simbolo = in programmazione è l'**operatore di assegnazione**: **copia** il valore che sta alla sua **destra** dentro alla **variabile** che sta alla sua **sinistra**.

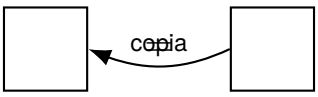


Figura 2: Schema alla lavagna per l'operatore di assegnazione: il valore nella scatola di destra viene copiato in quella di sinistra – la freccia va sempre da destra verso sinistra.

Esempio

Creiamo tre **variabili** di tipo `int`, con **nomi scelti liberamente** (ad esempio `dui`, `venom`, `calzino` – il nome non deve avere per forza un legame con quello che rappresenta, ma un **nome sensato** aiuta a rileggere il codice più tardi):

```
int dui;
int venom;
int calzino;
```

Non dimenticare il **punto e virgola** alla fine di ogni riga: chiude il comando.
Poi assegniamo dei valori:

```
dui = 7;
venom = 10;
```

```
calzino = dui - venom;
```

Qui `dui` contiene 7, `venom` contiene 10, e `calzino` riceve il **risultato** di `dui - venom`, cioè -3: il computer **esegue l'operazione** e copia il valore che ne esce dentro alla variabile a sinistra.

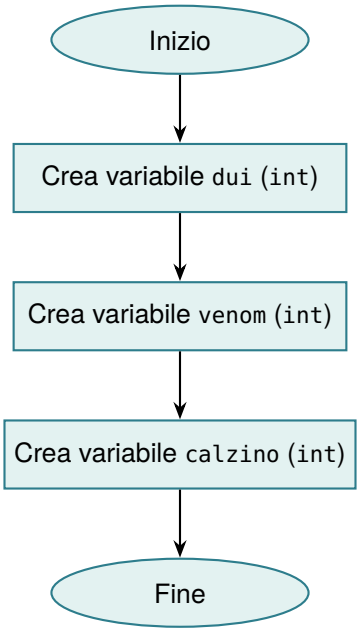


Figura 3: Diagramma di flusso: dichiarazione in sequenza delle tre variabili `int`.

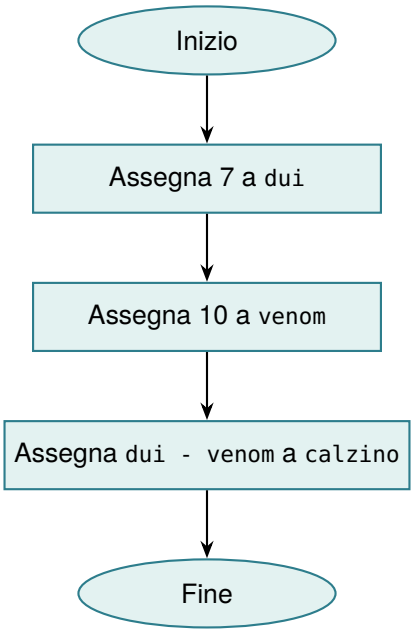


Figura 4: Diagramma di flusso: le tre assegnazioni eseguite in sequenza, nell'ordine scritto nel codice.

Nota

Attenzione allo **spazio** tra l'operatore e il numero: `dui = 7;` funziona esattamente come `dui=7;` (senza spazi) – il compilatore **non fa differenza** – ma la versione con gli spazi è più

leggibile, quindi va preferita.

Attenzione

Il simbolo = in programmazione **non è** il simbolo di **uguaglianza** della matematica. In matematica, "$x = 3$" mette in relazione x e 3, cercando per quale valore di x l'affermazione è vera. In programmazione, = *copia sempre da* ***destra verso sinistra***: scrivere `5 = a` non ha senso, perché a sinistra non c'è una **variabile** (un numero come 5 non può "**contenere**" un altro valore) – questo produce quasi sempre un **errore di compilazione**.

5 Le regole per i nomi delle variabili

Definizione

Il nome di una **variabile** deve rispettare queste regole:

1. È una sequenza **alfanumerica**: può contenere **lettere e numeri**.
2. Non può **iniziare con un numero** (puoi usare numeri, ma non come primo carattere).
3. **Non può contenere caratteri speciali**, a eccezione dell'**underscore** (`_`, **trattino basso**) – l'unica eccezione ammessa.
4. È **case sensitive**: `pippo` e `Pippo` sono due variabili ***diverse***.
5. **Non può coincidere con una parola riservata** del linguaggio (es. `int`, `char`, `return`, `for`): sono parole che il linguaggio C usa per sé, non si possono riusare come nome di variabile.

Esempio

Esempi fatti alla lavagna, con la classe chiamata a decidere se ogni nome fosse valido o no:

- `MICHELE` – **valido**.
- `pippo11` – **valido** (lettere+numero, non **inizia con un numero**).
- `12LUIGI` – **non valido**: **inizia con un numero**.
- `MAURO_BORTOLANI` – **valido**: l'**underscore** è l'unico carattere speciale ammesso.
- `MAURO BORTOLANI` (con uno **spazio**) – **non valido**: lo spazio interrompe la sequenza, "spezzando" il nome in due elementi distinti per il linguaggio.
- `a,b-5` – **non** valido: la **virgola** e il trattino sono **caratteri speciali** non ammessi.

Esempio

Case sensitive in pratica: `pippo`, `Pippo` e `Pippo0` sono **tre variabili diverse** agli occhi del **compilatore**, anche se a un lettore umano sembrano "**quasi la stessa cosa**".

Attenzione

Si può iniziare un nome con l'**underscore**, ma nel linguaggio C le variabili che iniziano così hanno un **significato tecnico** particolare (variabili di sistema/direttiva) che non useremo in questo corso – meglio evitarlo, per non creare confusione.

Esempio

Esercizio guidato in aula – dichiariamo tre **variabili** e proviamo a usarle in un calcolo con il modulo:

```
char ab5;
char z7b;
char 7b2;       // ERRORE: il nome inizia con un numero!

7b2 = 3;
z7b = 9;
ab5 = z7b % 7b2;
```

Il nome `7b2` viola la **regola 2** (non può iniziare con un numero): il programma **non compila**. Corretto togliendo il **numero iniziale**:

```
char ab5;
char z7b;
char b2;        // corretto: ora inizia con una lettera

b2 = 3;
z7b = 9;
ab5 = z7b % b2;    // ab5 riceve 9 % 3 = 0
```

6 Stampare un risultato a video: printf

Finora la macchina esegue i calcoli, ma non ce li mostra: serve un modo per *pubblicare* sul **monitor** il **risultato** delle nostre operazioni tramite il comando **printf**.

Definizione

Il comando **`printf`** (tutto minuscolo) **stampa** un testo sul monitor. Si scrive:

```
printf("testo");
```

Il testo va racchiuso tra **doppi apici** (`"`), dentro le parentesi tonde.

Esempio

Per stampare il **contenuto di una variabile** all'interno del testo, si usa uno **specificatore di formato**: il simbolo `%d` (percento-di), inserito nel punto in cui si vuole che **compaia il valore**.

```
printf("%d", dui);
```

Se la variabile `dui` vale -3, questo comando **stampa a video**: `-3`.

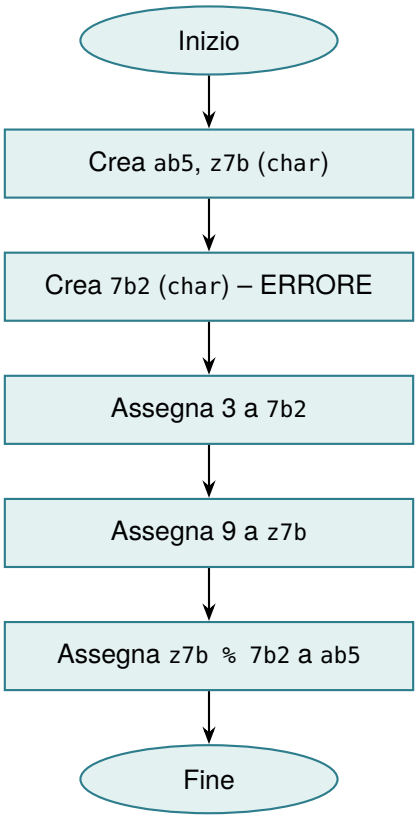


Figura 5: Diagramma di flusso della versione errata: la sequenza si interrompe alla compilazione per il nome `7b2` non valido.

Nota

Il simbolo `%` qui non ha niente a che fare con la **percentuale matematica**: introduce uno **specificatore di formato**, cioè un'**istruzione speciale** che dice "in questo punto metti il valore di una variabile", non un **carattere letterale** da stampare.
Da non confondere con le **sequenze di escape** (es. `\n` per **andare a capo**, `\t` per una tabulazione): quelle iniziano con il backslash `\` e rappresentano un **carattere speciale** nel testo, mentre `%d` dice a `printf` come **interpretare e mostrare** un valore – sono **due meccanismi diversi**, anche se si usano insieme dentro le stesse doppie virgolette.

Attenzione

Attenzione agli **spazi** e alla punteggiatura dentro `printf`: una **virgola dimenticata** o uno spazio sbagliato fanno fallire la compilazione. Un ambiente pratico per fare questi primi esperimenti senza installare nulla è onlinegdb.com: non è uno strumento professionale, ma è comodo per iniziare, funziona da qualunque browser (anche su tablet o smartphone).

7 Esercitazione pratica su onlinegdb.com

In questa lezione abbiamo scritto ed eseguito insieme un piccolo **programma**, partendo dal **codice di esempio** già presente nell'**ambiente online**. La parte che ci interessa è sempre quella compresa tra le due **parentesi graffe** della funzione **`main`** (cioè tutto quello che sta tra la graffa di

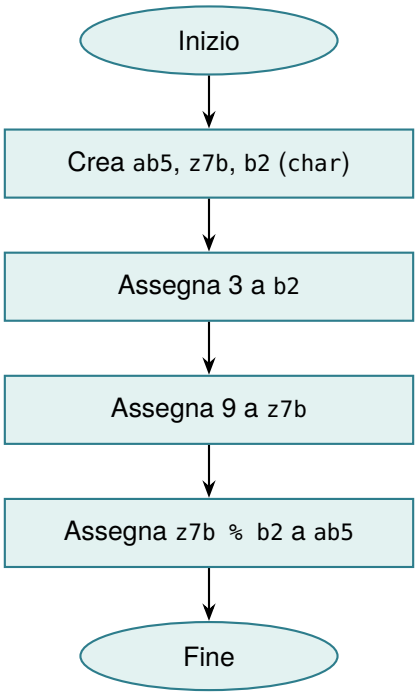


Figura 6: Diagramma di flusso della versione corretta: `ab5` riceve infine il valore $9 \bmod 3 = 0$.

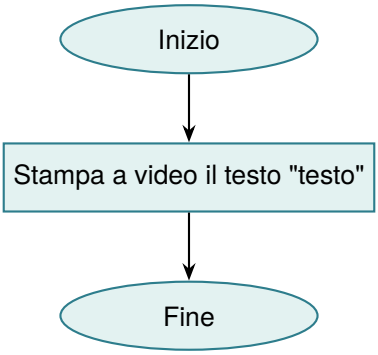


Figura 7: Diagramma di flusso: un'unica azione, la stampa del testo sul monitor.

apertura e quella di chiusura): lì dentro cancelliamo il contenuto di esempio e **scriviamo il nostro codice**.

Nota

Come scrivere le parentesi graffe su tastiera italiana: la graffa aperta `{` si ottiene con `Alt Gr + Shift + è` (il tasto con la doppia parentesi quadra); la graffa chiusa `}` con `Alt Gr + Shift +` il tasto accanto. Sono tra i simboli più usati in programmazione: vale la pena **memorizzare subito** la combinazione.

Attenzione

Errore comune: **dimenticare la graffa di chiusura** alla fine del programma produce un **errore di compilazione**. Se il programma non parte e non è chiaro perché, la prima cosa da controllare è che ogni **graffa aperta** abbia la sua **graffa di chiusura** corrispondente.

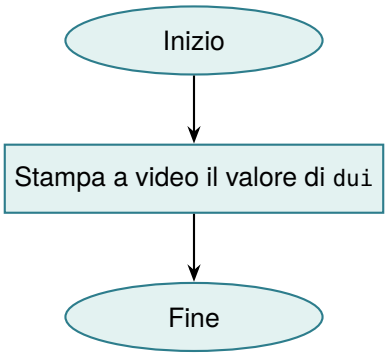


Figura 8: Diagramma di flusso: un'unica azione, la stampa del valore della variabile `dui` tramite `%d`.

Attenzione

Un problema più insidioso: se il browser ha attivato la ***traduzione automatica*** della pagina (ad esempio il traduttore di Google integrato), l'ambiente di programmazione può apparire con del testo tradotto in modo scorretto – ad esempio la **parola chiave** `return` potrebbe comparire tradotta e **non più riconosciuta** come parola del linguaggio C, causando errori strani e difficili da capire. Se questo succede, bisogna **disattivare la traduzione automatica** della pagina dalle impostazioni del browser.

Una volta scritto il codice, il pulsante **Run** (in alto, di colore verde) **esegue il programma**. Se compare un errore invece del risultato atteso, quasi sempre la causa è una delle seguenti: una **graffa dimenticata**, una **virgola** o un **punto e virgola mancante**, oppure un **nome di variabile** scritto in modo diverso da come è stato dichiarato (ricorda: il linguaggio C è **case sensitive**).

8 Cosa ricordare di questa lezione

- Gli **operatori aritmetici**: `+ - * /` (quest'ultimo si chiama **slash**).
- L'**operatore modulo** (`%`) dà il **resto** di una divisione tra interi, non il **risultato della divisione**.
- L'**operatore di assegnazione** (`=`) copia il valore di **destra** nella variabile di **sinistra** – **non** è l'**uguaglianza matematica**.
- Regole per i nomi di variabile: **alfanumerico**, non può **iniziare con un numero**, niente **caratteri speciali** (tranne **underscore**), **case sensitive**.
- `printf("%d", variabile);` **stampa a video** il valore di una variabile, usando lo **specificatore di formato** `%d`.
- Su tastiera italiana, le **graffe** si scrivono con `Alt Gr + Shift +` il tasto corrispondente – e vanno sempre in **coppia** (apertura/chiusura).

Esempio

Da fare per la prossima lezione: crea due variabili `char`, assegna loro due valori a scelta, calcola la **somma** e il **resto** della loro divisione (operatore modulo), e **stampa** entrambi i risultati con **`printf`**.